Arquitectura del Sistema Examio

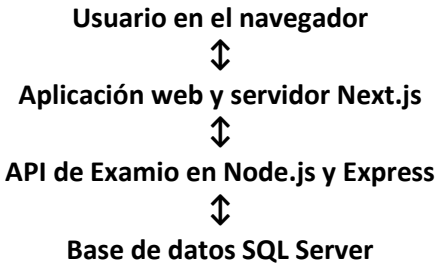
Una explicación de sus partes y de cómo trabajan juntas

7 de octubre de 2026

Examio es un sistema para administrar evaluaciones académicas. Permite organizar materias y alumnos, preparar exámenes, rendirlos, corregir respuestas y consultar resultados. Este documento explica cómo se conectan sus componentes y qué responsabilidad tiene cada uno.

Cómo se organiza el sistema

La aplicación web se apoya en una arquitectura por capas. La interfaz muestra la información y recibe las acciones del usuario; la API procesa las operaciones y controla los permisos; SQL Server guarda los datos y aplica restricciones de integridad. Next.js también aporta una parte de servidor que administra la sesión y sirve de puente hacia la API.



Las flechas representan solicitudes y respuestas. El navegador no se conecta directamente a la base de datos. Dentro del servidor Next.js, el puente hacia la API se llama BFF, abreviatura de Backend for Frontend.

Quiénes lo utilizan

Rol	Uso principal
Administrador	Gestiona usuarios, alumnos, docentes, carreras, materias y comisiones.
Docente	Prepara y publica evaluaciones, reutiliza preguntas, corrige y consulta reportes.
Alumno	Accede a sus evaluaciones, responde preguntas y consulta resultados.

Las partes de la arquitectura

La aplicación web

El frontend está construido con Next.js 16, React 19 y TypeScript. Next.js organiza las páginas y la navegación; React permite componer pantallas con componentes reutilizables; TypeScript ayuda a detectar incompatibilidades en los datos durante el desarrollo. Tailwind CSS se usa para los estilos.

Las rutas principales se separan por rol: /admin, /profesor y /alumno. Cada área reúne las funciones que necesita ese usuario. Los formularios validan la entrada para dar mensajes claros, pero la API vuelve a validar antes de aceptar una operación.

El código también se divide según su tarea. Las páginas están en src/app, los componentes en src/components, las consultas y actualizaciones en src/hooks, las llamadas HTTP en src/services/api y los contratos de datos en src/types. Las herramientas comunes, la sesión y el cliente HTTP están en src/lib.

El puente entre la web y la API

Las consultas de la interfaz pasan por hooks de TanStack React Query, servicios de dominio y un cliente común llamado apiFetch. Los hooks son funciones que conectan las pantallas con sus datos. React Query administra la carga, los errores y la caché, que permite reutilizar datos ya consultados.

apiFetch envía las solicitudes a /api/examio/*, dentro del mismo sitio. El BFF toma las credenciales de la sesión en el servidor y agrega la autorización al comunicarse con la API. Así, los componentes del navegador no necesitan manejar los tokens de acceso.

La API y la base de datos

La API REST está desarrollada con Node.js, Express 5 y TypeScript. Recibe solicitudes HTTP con datos en JSON, valida su contenido con Zod y comprueba el rol y los permisos del usuario. Sus rutas y controladores resuelven operaciones como publicar una evaluación, guardar una respuesta o consultar una nota.

La conexión con SQL Server se realiza mediante mssql, que reutiliza conexiones disponibles. La base GestionExamio guarda la información académica, las relaciones entre registros y parte de la lógica mediante procedimientos almacenados. Las operaciones que deben completarse juntas utilizan transacciones: si una parte falla, se revierten los cambios de esa operación.

Tecnologías de apoyo

NextAuth administra la sesión web. Zod valida formularios y contratos según la capa. Recharts dibuja los gráficos de los reportes. IndexedDB, un almacenamiento local del navegador, conserva respuestas de examen pendientes de sincronización. Cada herramienta cubre una necesidad concreta dentro del sistema.

Cómo se relacionan los datos

MateriaDivision es una entidad central: vincula una materia con una división o comisión y, cuando corresponde, con un docente. Las inscripciones determinan qué alumnos pertenecen a esa comisión y qué evaluaciones pueden consultar.

Concepto	Qué representa
Actividad	El contenido de una evaluación, trabajo práctico o tarea, con sus grupos de preguntas.
Publicación	La disponibilidad de una actividad para una comisión, con fechas y visibilidad.
Intento	La realización de una publicación por un alumno en particular.
Pregunta y opción	Los enunciados y las alternativas disponibles cuando el tipo de pregunta las requiere.
Respuesta	Lo que contestó el alumno en una pregunta de su intento.

Separar actividad, publicación e intento permite distinguir el examen preparado por el docente, su disponibilidad para una comisión y el trabajo realizado por cada alumno.

El recorrido de una evaluación

1. El docente prepara el contenido y lo guarda como borrador. Puede crear preguntas o traerlas desde su banco privado. Cada importación genera una copia independiente; los cambios posteriores no se sincronizan automáticamente con el original.
2. Cuando decide publicar, se envían el contenido final y la configuración a la API. La publicación completa es transaccional. Si el docente elige guardar todo en el banco, esa acción forma parte de la misma operación.
3. El alumno inicia un intento. El sistema selecciona una variante por grupo y registra esa selección en IntentoPregunta, de modo que el examen conserve las mismas preguntas al volver a consultarlo.
4. Al responder, la web guarda primero una copia local pendiente y luego intenta enviarla. Al recuperar la conexión, sincroniza las respuestas que faltan. Antes de entregar, debe confirmar que la cola pendiente quedó vacía.
5. La finalización calcula la parte automática de preguntas de opción múltiple y verdadero o falso. Las respuestas que requieren corrección manual quedan a cargo del docente; después, los resultados alimentan las consultas y los reportes.

Para evitar publicaciones duplicadas ante reintentos, se utiliza una clave de idempotencia: identifica la misma operación aunque se envíe otra vez. Las revisiones del banco y los borradores permiten detectar si otro cambio dejó desactualizada la versión que se está editando.

Sesión y cuidado de las respuestas

El usuario ingresa con correo y contraseña. NextAuth consulta a la API y mantiene la sesión en una cookie cifrada y HttpOnly. Esto impide que el código JavaScript de la página lea directamente las credenciales guardadas allí. La información de sesión expuesta a la interfaz contiene la identidad y el rol, sin los tokens de la API.

La web controla la navegación según el rol, pero la autorización definitiva se realiza en la API. Por ejemplo, ocultar un botón de edición no alcanza: el servidor debe rechazar la operación si el usuario no tiene permiso.

Durante un examen, las respuestas pendientes se conservan en una cola local, llamada outbox. Si se pierde la conexión, la interfaz mantiene el intento visible y diferencia lo guardado de lo pendiente. Si IndexedDB no está disponible, el respaldo queda en memoria y se advierte que no debe cerrarse ni recargarse la pestaña. Este mecanismo ayuda a conservar respuestas; no convierte toda la aplicación en un sistema sin conexión.

La aplicación móvil

Examio Native complementa el proyecto con React Native, Expo y TypeScript. Usa Expo Router para navegar y Zustand para mantener el usuario en memoria. Su objetivo es que el alumno consulte desde el celular próximas evaluaciones, fechas y calificaciones.

En su estado actual tiene un inicio de sesión simulado y una pantalla de próximas actividades con datos estáticos. Todavía no está conectada con la API y la sesión no se conserva al cerrar o recargar por completo la aplicación. Los recordatorios, el historial de notas y las notificaciones figuran como funciones pendientes.

Pruebas y estado actual

El frontend cuenta con Vitest y React Testing Library para probar lógica y componentes, MSW para simular respuestas HTTP y Playwright para recorrer flujos en un navegador. Los recorridos automatizados usan una API de prueba local, sin modificar datos académicos reales.

La web ya integra el banco privado, los borradores, la publicación y los reportes. Siguen pendientes mejoras en la gestión de etiquetas y en la presentación de conflictos de edición. En los reportes, los gráficos por alumno representan la página visible; los del reporte por evaluación usan agregaciones globales de la API.

Documentación de referencia

Para ampliar: AGENTS.md y README.md del frontend; docs/API.md para la API; docs/DATABASE.md para las entidades; PLAN.md para el trabajo pendiente; y README.md de examio-native para el alcance móvil. Este documento describe la organización lógica del sistema, sin definir una infraestructura de producción concreta.